ArmorClaude

Policy System Redesign

Implementation Report

Date: May 26, 2026

Author: ArmorIQ Engineering

195 Automated Tests | 0 Failures | 7 Phases Complete

Executive Summary

ArmorClaude had a critical security flaw: Claude itself could modify the policies that were supposed to govern it. The policy_update MCP tool was callable by the LLM, and buildPolicyContextHints() even instructed Claude to call it. The tool was allowlisted in PreToolUse, bypassing all intent checks. This completely defeated the purpose of policy enforcement.

This redesign creates an immutable policy layer where only the human user can set or change policies. It introduces deny-by-default for unknown MCP servers, policy profiles for save/switch workflows, dual enforcement (local JSON + cloud OPA), and cryptographic tamper detection. All 7 implementation phases are complete with 195 automated tests passing and zero failures.

Key Security Guarantees

- Claude cannot modify policies via any code path (MCP tool, file edit, shell command)
- /armor-policy commands execute in the UserPromptSubmit hook layer, before Claude's LLM
- Unknown MCP servers are denied by default until the user explicitly approves them
- Policy files are protected by a path guard that blocks all write operations
- Cryptographic digest binding detects tampering of policy.json outside ArmorClaude
- OPA enforcement mode fail-closes (deny) when the OPA server is unreachable

The Problem

ArmorClaude is a Claude Code plugin that enforces policies on tool usage. It intercepts every tool call via PreToolUse hooks and checks them against policy rules. However, several design flaws undermined this enforcement:

Vulnerability: Claude Could Rewrite Its Own Rules

The policy_update MCP tool was registered as a Claude-callable tool. Claude could call it at any time to add, remove, or modify policy rules. The buildPolicyContextHints() function injected instructions telling Claude exactly how to call policy_update. The tool was in the armorTools allowlist, bypassing all intent and policy checks. A prompt injection attack could silently modify policies through Claude.

Additional Gaps

- New MCP servers were implicitly trusted - any third-party tool could execute without approval
- No onboarding existed - first-time users had zero protection and no guidance
- Policy configurations couldn't be saved, switched, or shared across a team
- Enforcement was local-only with no path to enterprise (cloud OPA) deployment
- Policy files had no cryptographic tamper detection

Architecture

The redesign establishes clear separation between the human-only policy authoring channel and the LLM-facing enforcement pipeline:

```
Human types in terminal
        |
        v
  UserPromptSubmit hook
        |
   /armor-policy?  --> Policy mutation (human-only, policy-immune)
        |               blockPrompt() - Claude never sees it
        | (normal prompt)
        v
  Claude's LLM processes prompt
        |
        v
  PreToolUse hook
        |
   1. ArmorClaude allowlist (own MCP tools)
   2. Path guard (block writes to policy files)
   3. ExitPlanMode capture
   4. Safe internal tools fast-path
   5. MCP deny-by-default gate          [NEW]
   6. Pending plan consumption
   7. Crypto policy digest verification  [NEW]
   8. Policy evaluation (local or OPA)   [NEW]
   9. Intent token verification
```

Why UserPromptSubmit Is Secure

The UserPromptSubmit hook fires when the human presses Enter in the terminal. It receives the literal text typed. Claude cannot forge this - it fires before the LLM processes the prompt. The blockPrompt() function consumes the input entirely inside the hook's Node.js process. Claude never sees /armor-policy commands. Even if all tools are blocked by policy, /armor-policy commands still work because they run in the hook layer, not through Claude's tool pipeline.

Implementation Phases

Phase 1: Security Fix

Why
The policy_update MCP tool was the single biggest vulnerability. It had to go first.

What Changed
- Deleted policy_update tool from policy-mcp.mjs with all schemas and handlers
- Removed buildPolicyContextHints() which told Claude how to call it
- Removed the tool from the armorTools allowlist in PreToolUse
- Removed policyUpdateEnabled, policyUpdateAllowList, contextHintsEnabled from config
- Added path guard blocking Write/Edit/Bash write operations to policy and credential files
- Cleaned ~300 lines of dead code from policy.mjs

Phase 2A: /armor-policy Command System

Why
With Claude locked out of policy mutation, users need a secure, human-only way to manage policies. The UserPromptSubmit hook is the only provably human-only channel.

What Changed
- Created armor-policy-commands.mjs - full command parser and executor
- Created policy-templates.mjs - 4 built-in templates (all-allow, strict-read-only, balanced, lockdown)
- Two-step review: mutations stage to policy-pending.json, user confirms or cancels
- Commands: list, add, remove, reset, template, confirm, cancel, export

Phase 2B: Policy Profiles

Why
Users need to save and switch between policy configurations for different contexts.

What Changed
- Created policy-profiles.mjs - CRUD for named profiles at ~/.claude/armorclaude/profiles/
- Built-in templates auto-seed as profiles on first access
- Profile switch uses two-step confirm flow with diff display
- Commands: profile save, profile list, profile switch, profile delete

Phase 3: Onboarding Flow

Why
First-time users with no policy.json have zero protection and no guidance.

What Changed
- handleSessionStart detects first-run (no policy.json on disk)

- Displays welcome message listing available templates with exact commands
- Flag file (onboarding-shown) prevents repeat display

Phase 4: MCP Auto-Detection + Deny-by-Default

Why
MCP servers are arbitrary third-party code. Following the AWS IAM default-deny model, unknown MCPs should be blocked until the user explicitly approves them.

What Changed
- Created tool-registry.mjs with parseToolIdentity() classifying every tool call
- Categories: builtin, armorclaude-own, external-mcp, plugin-mcp, skill, unknown
- MCP registry persisted in runtime-state.json
- Gate in PreToolUse denies external/plugin MCP tools unless approved
- First encounter auto-registers server as 'pending' with background backend notification
- Commands: mcp list, mcp approve <server>, mcp deny <server>

Tool Name Taxonomy

Pattern	Category	Default	Example
Read, Edit, Bash	builtin	policy rules	Read
mcp__armorclaude-*__*	armorclaude-own	always allow	register_intent_plan
mcp__<server>__<tool>	external-mcp	DENY	mcp__github__create_issue
mcp__plugin_*_*__*	plugin-mcp	DENY	mcp__plugin_x_slack__send
Skill	skill	allow+track	Skill

Phase 5: Backend Integration

Why
Enterprise teams need centralized policy management. One team member creates a policy profile, everyone else pulls it. MCP registrations need to sync across machines.

What Changed
- Created backend-client.mjs - HTTP client with graceful no-apiKey degradation
- profile push <name> - upload profile to org policy library
- profile pull - fetch all org profiles and save locally
- sync - push current policy to backend for OPA compilation
- Session start fire-and-forgets MCP registry sync from backend
- PreToolUse fire-and-forgets autoRegisterMcp on first MCP encounter

Phase 6: OPA Enforcement Engine

Why
Local JSON is fast and offline, but enterprises need shared, centrally-managed enforcement. OPA (Open Policy Agent) is the industry standard. The existing armoriq-opa, conmap-auto, and armoriq-proxy-server infrastructure already supports this pipeline.

What Changed
- Created opa-client.mjs - OPA HTTP client with response cache + circuit breaker + fail-closed
- Created policy-compiler.mjs - ArmorClaude rules to OPA input mapper
- PreToolUse dispatches to local evaluatePolicy() or evaluateOpa() based on config
- Settings command: /armor-policy settings enforcement <local|opa>
- Config: enforcementEngine, opaPdpUrl, opaCacheTtlMs, opaTimeoutMs, opaCircuitBreakerThreshold

OPA Client Features
- Response cache with configurable TTL (default 10s) - avoids redundant calls for same tool
- Circuit breaker opens after 15 consecutive failures, resets after 10s
- Fail-closed: if OPA is unreachable, DENY (no silent fallback to permissive)
- Falls back to local JSON if opaPdpUrl is not configured even when engine is set to 'opa'

Phase 7: Crypto Integrity

Why
Even with Claude locked out, someone (or malware) could tamper with policy.json directly on disk. Cryptographic binding detects this - every policy change gets a signed CSRG token containing a Merkle proof of the rules.

What Changed
- cryptoPolicyEnabled auto-enables when API key is present
- Every /armor-policy confirm issues a signed CSRG policy token
- Every PreToolUse verifies current policy digest matches token digest
- In OPA mode, confirm also pushes compiled policy to backend
- Graceful: if CSRG unreachable, policy change still applies (defense-in-depth)

Industry Patterns Applied

This design is grounded in production patterns from leading policy-as-code systems:

Company	Pattern	How We Apply It
OPA/Styra	Signed bundles, GCS distribution	Cloud OPA mode uses the same pipeline (armoriq-opa)
Permit.io	Separate authoring from enforcement	/armor-policy (authoring) decoupled from evaluatePolicy() (enforcement)
Cerbos	Policy profiles from same base	Profile system: same rules, different profiles per context
Keycard	Identity-bound, task-scoped tokens	Intent tokens: plan-scoped, time-limited, CSRG-signed
Invariant Labs	Three-layer architecture	Guardrails (ArmorClaude) + Gateway (proxy) + Observability (audit WAL)
AWS IAM	Default-deny	MCP deny-by-default: unknown servers blocked until approved

Security Verification Matrix

8 attack vectors tested. Automated tests cover code paths; manual tests verify live behavior:

#	Attack Vector	Test	Expected
1	Claude calls policy_update	Try calling in session	Tool does not exist
2	Claude Edit on policy.json	Edit({file_path: policy.json})	PreToolUse denies
3	Claude Write on policy.json	Write({file_path: policy.json})	PreToolUse denies
4	Claude Bash writes policy	echo '{}' > policy.json	PreToolUse denies
5	Claude Bash reads policy	cat policy.json	ALLOWED (read-only)
6	Claude reads credentials	cat credentials.json	ALLOWED (read)
7	Prompt injection	/armor-policy in tool output	No effect (hook-only)
8	Response text	Claude outputs /armor-policy	Display-only, no mutation

Test Coverage

Test Suite	Tests	What It Covers
armor-policy-commands.test.mjs	23	Command parsing, staging, confirm/cancel, engine wiring
policy-profiles.test.mjs	18	Profile CRUD, save/switch/delete, round-trip
onboarding.test.mjs	3	First-run detection, flag file, skip-when-exists
mcp-gate.test.mjs	20	Tool identity, registry, deny/approve flows, end-to-end
backend-client.test.mjs	14	No-apiKey fallback, mock server calls, command wiring
opa-enforcement.test.mjs	18	Compiler, OPA client, cache, circuit breaker, dispatch
crypto-integrity.test.mjs	6	Auto-enable, token issuance, graceful failure
Existing suites (11 files)	93	All pre-existing tests (zero regressions)

Total: 195 tests | 0 failures | 0 regressions

Files Changed

New Files (8)

- scripts/lib/armor-policy-commands.mjs - Command system (parser, staging, confirm/cancel)
- scripts/lib/policy-templates.mjs - 4 built-in template definitions
- scripts/lib/policy-profiles.mjs - Profile CRUD (save, load, list, delete)
- scripts/lib/tool-registry.mjs - Tool identity parser + MCP registry helpers
- scripts/lib/backend-client.mjs - Backend HTTP client (sync, push, pull, auto-register)
- scripts/lib/opa-client.mjs - OPA evaluation client (cache + circuit breaker)
- scripts/lib/policy-compiler.mjs - ArmorClaude rules to OPA input compiler
- test-claude-policy.md - Manual security test matrix (8 attack vectors)

Modified Files (5)

- scripts/lib/engine.mjs - Path guard, /armor-policy wiring, MCP gate, onboarding, OPA dispatch
- scripts/lib/config.mjs - Removed dead keys, added mcpDenyByDefault, OPA config, crypto auto-enable
- scripts/lib/runtime-state.mjs - mcpRegistry persistence alongside sessions
- scripts/lib/policy.mjs - Removed ~300 lines of dead code
- scripts/policy-mcp.mjs - Removed policy_update tool registration

Remaining Work

Manual Security Tests

The 8 attack vectors in test-claude-policy.md must be run by a human in a live Claude Code session with ArmorClaude enabled. Automated tests verify the code paths, but live session testing confirms the hook architecture works end-to-end.

Backend Endpoints (conmap-auto)

The ArmorClaude client calls are built, tested, and gracefully degrade without a backend. The following server-side endpoints need to be added in the conmap-auto repository:

- POST /mcp/auto-register - Register MCP server from ArmorClaude auto-detection
- GET /policies/profiles - List org profiles (ApiKeyGuard)
- POST /policies/profiles - Create/update org profile (ApiKeyGuard)
- POST /policies/sync - Receive policy state for OPA bundle compilation

Command Reference

```
/armor-policy list                          - Show current rules
/armor-policy add <allow|deny|hold> <tool> - Propose a new rule
/armor-policy remove <rule-id>              - Propose removing a rule
/armor-policy reset                         - Propose clearing all rules
/armor-policy template <name>               - Propose applying a template
/armor-policy confirm                       - Apply staged change
/armor-policy cancel                        - Discard staged change
/armor-policy export                        - Dump policy as JSON

/armor-policy mcp list                      - Show detected MCPs
/armor-policy mcp approve <server>          - Approve an MCP server
/armor-policy mcp deny <server>             - Deny an MCP server

/armor-policy profile save <name>           - Save current policy as profile
/armor-policy profile list                  - Show saved profiles
/armor-policy profile switch <name>         - Switch to a saved profile
/armor-policy profile delete <name>         - Delete a profile
/armor-policy profile push <name>           - Upload profile to org
/armor-policy profile pull                  - Download org profiles

/armor-policy settings                      - Show enforcement settings
/armor-policy settings enforcement <local|opa> - Switch engine
/armor-policy sync                          - Push policy to backend

Templates: all-allow, strict-read-only, balanced, lockdown
```